MORTAR: Multi-Turn Metamorphic Testing for LLM-Based Dialogue Systems

Guoxiang Guo, Aldeida Aleti, *Associate Member, IEEE*, Neelofar Neelofar, Chakkrit Tantithamthavorn, *Senior Member, IEEE*, Yuanyuan Qi, and Tsong Yueh Chen, *Life Fellow, IEEE*

***Abstract*—With the widespread application of LLM-based dialogue systems in daily life, quality assurance has become more important than ever. Recent research has successfully introduced methods to identify unexpected behaviour in single-turn testing scenarios. However, multi-turn interaction is the common real-world usage of dialogue systems, yet testing methods for such interactions remain underexplored. This is largely due to the oracle problem in multi-turn testing, which continues to pose a significant challenge for dialogue system developers and researchers. In this paper, we propose MORTAR, a metamorphic multi-turn dialogue testing approach, which mitigates the test oracle problem in testing LLM-based dialogue systems. MORTAR formalises multi-turn testing for dialogue systems, and automates the generation of question-answer dialogue test cases with multiple dialogue-level perturbations and metamorphic relations (MRs). The automated MR matching mechanism provides MORTAR more flexibility and efficiency in testing. When applied to eight LLM-based dialogue systems as test objects, MORTAR reveals 51% more bugs per test case than the best-performing single-turn metamorphic testing baseline, with 24.6% higher precision and over 42% of detected bugs being unique.**

***Index Terms*—Metamorphic testing, dialogue system testing, SE4AI.**

I. Introduction

RAPID development of large language models (LLMs) has led to substantial capability improvements in downstream applications. LLM-based dialogue systems e.g., the closed-source LLM-based dialogue system ChatGPT [1], and the open-sourced LLM-based dialogue system Ollama [2], are now frequently used by real-world users in their daily lives. However, numerous unexpected behaviours are observed in the outputs, e.g., incorrect information and irrelevant answers [3]. Comprehensive testing is essential to ensure the output quality of LLM-based dialogue systems. However, testing such natural language generation systems inevitably encounters the oracle problem [4]. In software testing, the test oracle is crucial, as it determines whether the system under test behaves as expected.

Traditionally, the development of test oracles relies on the experience of system testers and the design of test suites. Crowdsource workers also play an important role in producing dialogue datasets as test cases. Many benchmark datasets [5], [6] can be used in reference-based testing (RBT) to simulate user input and trigger the unexpected behaviours of LLMs. The form of existing testing can be further categorised into single-turn and multi-turn. Single-turn testing involves presenting the LLM with a standalone input prompt and evaluating the quality of its immediate response. Multi-turn testing is conducted through multiple back-and-forth rounds, which simulate real-world dialogue between the user and LLM. It is worth noting that most existing testing datasets and approaches only provide single-turn testing capability [3]. According to real-world usage, however, over 63% of dialogues contain more than two rounds [7]. Such a mismatch between system testing and actual usage can lead to critical issues [8].

The main challenge of multi-turn testing is the automation of test oracles. The ideal test oracle is human judgement, but it is inevitably costly and inaccessible in testing scenarios where time and resources are constrained. Numerous testing approaches rely on LLM judges to give the final verdict[9], [10], while prompting an LLM to provide a pass or fail verdict inevitably introduces bias, and further harms the reliability of evaluation [11]. Considering the gap in existing approaches, which primarily focus on single-turn interactions, and the multi-turn nature of real-world systems, this research aims to propose an automated multi-turn dialogue testing approach that decouples the test oracle from straightforward verdicts of LLM judgements.

Metamorphic testing (MT) [12], a well-established software testing method, has shown the potential to alleviate the oracle problem in the testing of generative AI (GenAI) systems [4]. Several MT-based approaches have been introduced to support single-turn testing for question-answering (QA) systems and large language models (LLMs) [13], [14], [15]. Perturbations in character level, word level, and sentence level show

Received 3 November 2025; revised 16 April 2026; accepted 28 May 2026. Date of publication 8 June 2026; date of current version 21 September 2026. Recommended for acceptance by H. Zhong. *(Corresponding author: Aldeida Aleti.)*

Guoxiang Guo, Aldeida Aleti, Chakkrit Tantithamthavorn, and Yuanyuan Qi are with the Faculty of Information Technology, Monash University, Clayton, VIC 3800, Australia (e-mail: Guoxiang.Guo@monash.edu; Aldeida.Aleti@monash.edu; Chakkrit@monash.edu; Yuanyuan.Qi@monash.edu).

Neelofar Neelofar is with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: neelofar.neelofar@rmit.edu.au).

Tsong Yueh Chen is with the School of Science, Computing and Emerging Technologies, Swinburne University of Technology, Hawthorn, VIC 3122, Australia (e-mail: tychen@swin.edu.au).

This article has supplementary downloadable material available at https://doi.org/10.1109/TSE.2026.3701230, provided by the authors.

Digital Object Identifier 10.1109/TSE.2026.3701230

0098-5589 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

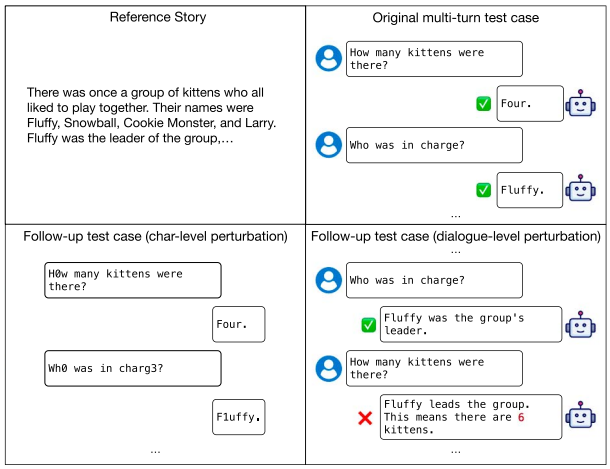


Fig. 1. Testing with different test cases. The test seed is the original test case with multiple question rounds. Dialogue-level perturbation (lower right) alters the context of question and the follow-up test case trigger a failure.

effectiveness in some testing scenarios. However, relying solely on single-turn perturbations in multi-turn dialogue testing fails to account for the essential characteristic of multi-turn interactions, as these single-turn perturbations are insufficient to verify the context dependence intrinsic to multi-turn dialogues. In Fig. 1, with the original test case as a test seed, dialogue-level perturbation produces test cases with nuanced context information for the same turn-wise input and reaches significantly enlarged dialogue coverage.

In this paper, we focus on general multi-turn QA dialogues, in which each question has a single factual answer, and the multi-turn interactions are used to clarify or decompose the information needed to reach that answer [16]. Under this scope, we propose MORTAR, a metamorphic multi-turn dialogue testing approach, which mitigates the oracle problem in the testing of LLM-based dialogue systems. Following the definition of existing works [14], [17], we regard the unexpected output of system under test as a bug, i.e., violation of MR in MT. Using the open-domain multi-turn dialogue dataset [16], MORTAR generates follow-up test cases to implement multi-turn MT for LLM-based dialogue systems. MORTAR formalises four MRs and proposes seven dialogue-level perturbations, using the multi-turn MT framework to generate follow-up test cases and reveal bugs in dialogue systems. The matching of MR is not conducted manually, but can be automated with an equivalent context check process, such that MORTAR is fully automated. According to the experimental results on multiple LLM-based dialogue systems, MORTAR achieves higher effectiveness in testing. Particularly, MORTAR reveals 164% more bugs than the most effective single-turn MT baseline [15]. Regarding the quality of bugs, MORTAR achieves better performance in terms of diversity, precision and uniqueness. The ablation studies verify the contribution of MRs and perturbations to the overall effectiveness. Implementation of MORTAR is made public[1].

The main contributions of this paper are as follows:

1) We formalise the framework of multi-turn metamorphic testing, which serves as the foundation of MORTAR.
2) Three basic and four derivative dialogue-level perturbations, and four MRs are proposed to reveal bugs in LLM-based dialogue systems.
3) To enable full automation of the testing process, the effect of each perturbation on the dialogue context is identified by an equivalent context check, which drives the selection of the proper MR for violation detection.
4) MORTAR successfully reveals more bugs in different dialogue systems regarding quantity, diversity, precision and uniqueness. Experiment results show that MORTAR outperforms the most effective single-turn metamorphic testing approach in identifying unique bugs.

The rest of this paper is structured as follows: Section II introduces the background and motivation. Section III elaborates on the details of MORTAR. Section IV describes the settings of experiments. Section V presents the experiment results analysis and discussion. Section VI reports threats to validity. Finally, Section VII introduces relate works and Section VIII concludes and discusses potential future works.

II. Background and Motivation

This section provides the necessary background to contextualise our work, covering key areas including the current landscape of testing LLM-based dialogue systems, the role of MT in addressing oracle challenges, and the critical distinction between single-turn and multi-turn testing approaches.

A. Testing and Evaluation of LLM-Based Dialogue Systems

Dialogue systems are now broadly applied in many domains, especially after the recent rapid development of LLMs [18], [19]. To ensure the quality of natural language generation applications, multiple datasets and evaluation methods emerged to test the dialogue system from many different aspects [6], [20], [21]. The test datasets typically comprise test input utterances and the expected outputs. Given a certain dialogue context and an input utterance, a bug detection is performed to judge if the output of the dialogue system meets the correctness criteria. If not, the detection will be recorded as positive to indicate a bug in the dialogue system. It is the common practice to test using such a reference-based scheme [22].

The testing of dialogue systems can be broadly categorised into single-turn and multi-turn. In single-turn testing, the system is provided with one input utterance at a time, and its response is evaluated against expected behaviour [15]. In contrast, multi-turn testing involves a sequence of inputs and outputs, simulating real conversational flows where the system must maintain context, coherence, and consistency across multiple exchanges [23]. Most existing approaches for testing LLM-based dialogue systems are single-turn.

Single-turn testing faces several well-known challenges. One major issue is the difficulty and high cost associated with acquiring reliable testing datasets [13]. Additionally, there are growing concerns about potential data contamination in existing

[1]https://github.com/Guoxiang365/MORTAR and https://zenodo.org/records/19556799

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

LLM training datasets [24], raising questions about the continued effectiveness of tests based on such datasets. These challenges are amplified in multi-turn testing. A multi-turn test case is a series of simulated user inputs of multiple utterances with corresponding expectations on output. High-quality multi-turn dialogue test datasets are significantly less available than their single-turn counterparts [25]. A recent survey [3] found that only 2 out of 46 evaluations focus on multi-turn interactions.

To mitigate the oracle problem, prompting LLMs to generate test datasets or score the outputs of system-under-test is adopted in an increasing number of studies. Recent multi-turn testing focuses on using LLMs to generate test cases and invite LLMs to function as judges to score the output of LLM-based dialogue systems [9], [10], [26], [27], [28]. The concern of using LLMs to generate test cases for LLM-based dialogue system testing is that LLMs are trained on massive datasets to converge and act normally, but testing requires test cases to be unfamiliar and diversified. Using LLM to generate test cases and evaluate the outputs faces issues such as the lack of diversity of test cases, hallucination, and bias [11]. Besides, these approaches generally focus on specific aspects, e.g., human-likeness [28] or dialogue capability benchmark [9], [26], and omit the pass/fail test oracle and criteria on the same test seeds, resulting in limited comparability with broader dialogue system testing methods. These factors lead to the unsatisfactory multi-turn testing of LLM-based dialogue systems. There exists a notable gap between current testing approaches and expectations of LLM-based dialogue systems. This paper introduces a multi-turn dialogue testing approach based on metamorphic testing. The usage of LLM in MORTAR is limited to constrained natural language processing task rather than adopting straightforward LLM judges to give end-to-end verdicts.

B. Metamorphic Testing

RBT relies on dataset annotation as the test oracle to judge the quality of output, and it is not feasible if the test oracle is inaccessible or no longer effective. MT was first proposed by Chen et al. in 1998 [12] to test numerical programs without reliance on test oracles. Over the past development, MT has been widely used to mitigate the oracle problem in software testing, especially for those low-resource testing scenarios with limited availability of testing datasets [29]. The fundamental idea of MT is to formalise MRs and generate follow-up test cases from the *test seeds*, then check if the output of the system with follow-up input violates the MRs [30]. The violation will be regarded as a system bug.

According to recent research, MT has been widely used in both testing traditional software systems and machine learning software systems [31], [32]. Test seed is the original test case that can be used to produce follow-up test cases. In testing dialogue systems, MT is promising in exploring more unique bugs with limited testing seeds [13], [14], [15]. However, there is no best practice for applying MRs in multi-turn dialogue testing. Previous testing is generally carried out in a single-turn manner where the perturbations applied to the dialogue system are low-level, including character-level, word-level, and sentence-level [33], [34].

Considerable single-turn MT methods are not applicable to multi-turn scenarios. For example, QAQA [14] is one of the latest single-turn MT approaches for dialogue systems. It splits the testing dataset into two parts and uses one as a searching space to find applicable perturbation elements and adds them to the target question. In multi-turn testing, QAQA is not feasible as: a) many questions in dialogue are too short to accurately calculate semantic of an individual short sentence that is unclear, which makes searching for related content infeasible; b) QAQA expects the candidate utterance from searching space to be self-contained and feasible to be injected into the target question, yet considerable questions in multi-turn dialogue are context-sensitive. Thus they cannot be inserted into other dialogues, and the dataset itself can not be used as a reliable searching space.

Another major difference between single-turn and multi-turn MT is that, for single-turn MT, perturbations are generally predefined to be either semantic-preserving or semantic-altering, and they will be manually matched with corresponding MR and then be used in MT [13], [14], [15], [33]. In multi-turn MT, the effect of dialogue-level perturbations might need additional processing to judge the actual effect on the test seeds. It is necessary yet difficult to tell whether a target question in the perturbed test case is given sufficient information from context to be answerable. This brings an additional challenge to multi-turn MT. Dialogue-level perturbation and MR have not been proposed in the prior work for its intrinsic difficulty in modelling and implementation.

In summary, realising multi-turn MT for dialogue system remains challenging for the following reasons:

- Relatively limited availability of multi-turn test seeds
- Absence of formalised multi-turn MRs
- Lack of dialogue-level perturbations
- Difficulty in test automation

To our knowledge, we are the first to systematically introduce dialogue-level MRs with implementation of effective and automated metamorphic test case generation pipelines and MR violation detection to mitigate the oracle problem in multi-turn dialogue system testing.

III. Approach

In this section, we introduce the framework of MORTAR, the formalised MRs and perturbations, and a feasible design of test automation.

A. Overview of MORTAR

As shown in Fig. 2, given the original dialogue dataset, MORTAR first generates the perturbed follow-up test input with three fundamental and four derivative dialogue-level perturbations (Section III-C). Second, the equivalent context check verifies if the context of each question in the perturbed test input is equivalent to the original context. In each input turn, the equivalent context check result will be used to match each question with the proper MR for bug detection (Section III-D). Finally, after running follow-up tests on LLM-based dialogue

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

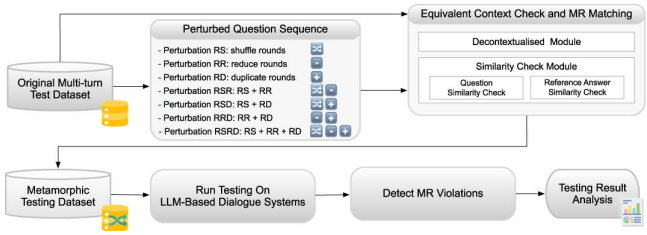


Fig. 2. An overview workflow of MORTAR.

systems, MORTAR detects violations of MRs in dialogue system outputs. Ideally, if the dialogue systems can pass the RBT with original test cases, they are expected to pass MORTAR as well. In this way, the original test cases are reused as the test seeds, and MORTAR generate metamorphic test cases to further test the dialogue systems.

B. Formalising Multi-Turn Metamorphic Testing Framework

In MORTAR, we focus on the open-domain general multi-turn QA testing scenarios where the correct answer to each question is unique under certain context. Each piece of dialogue D, it is composed of n question answer pairs:

$$D = [(q_1, a_1), (q_2, a_2), \ldots, (q_n, a_n)] \tag{1}$$

where the ordering of (q_i, a_i) matters, and all questions q_i and answers a_i in D can be regrouped as two lists, Q and A:

$$Q = [q_1, q_2, \ldots, q_n], \tag{2}$$
$$A = [a_1, a_2, \ldots, a_n]. \tag{3}$$

We use subscripts on lists to denote the initial subsequence:

$$D_i := [(q_1, a_1), \ldots, (q_i, a_i)], \tag{4}$$
$$Q_i := [q_1, \ldots, q_i], \tag{5}$$
$$A_i := [a_1, \ldots, a_i], \tag{6}$$

where $i \in \{1, \ldots, n\}$, D_i is the first i elements of D, similarly for Q_i and A_i, such that D_{i-1} can be regarded as context information when the dialogue system receives q_i, and the context of q_1 is empty, i.e. $D_0 = []$. An ideal dialogue system, $IDS(\cdot)$, exhibits the following behaviour:

$$IDS(D_{i-1}, q_i) \to a_i. \tag{7}$$

Equation (7) represents questions that require contextual information, which is common in multi-turn dialogue. The user usually refers to information or requirements given in context with pronouns or other types of ellipsis [35] for simplicity and convenience. However, if the contextual dependence is disrupted by perturbations and the critical information is missing in context, the target question shall not be answered with the original answer. In other cases, the target question remains context equivalent if: a) the redundant or unrelated rounds to the target question are removed by perturbations; or b) the target question can be answered independently without reliance on the context, e.g., the first question in original dialogue (q_1), which is usually information-complete and can be answered under empty context.

To take a close look at the context dependency in Equation (7), the context of q_i in input, D_{i-1}, is composed of the previous context D_{i-2}, the question q_{i-1}, and the answer a_{i-1}. The answer a_{i-1} in the previous round is further produced by earlier round:

$$IDS(D_{i-2}, q_{i-1}) \to a_{i-1}. \tag{8}$$

Such that the information in a_{i-1} has been implied by question q_{i-1} and its context D_{i-2}. Equation (7) can be rewritten as:

$$IDS([D_{i-2}, q_{i-1}], q_i) \to a_i. \tag{9}$$

Similarly, since all previous answers are dependent on the sequence of questions in the corresponding context, Equation (9) can be expressed as:

$$IDS([q_1, \ldots, q_{i-1}], q_i) \to a_i. \tag{10}$$

Or, after the definition of Q_{i-1}, as follows:

$$IDS(Q_{i-1}, q_i) \to a_i. \tag{11}$$

It is intuitive that the answer can be regarded as implied in the question sequence of context.

Conducting RBT on a dialogue system $DS(\cdot)$ with the original test case, is operated as feeding the dialogue system with question sequence Q then verifying the quality of each output. Given the context question sequence Q_{i-1} and the target question q_i, the dialogue system generates responses o_i:

$$DS(Q_{i-1}, q_i) \to o_i. \tag{12}$$

It is expected that the dialogue system under test generate similar answers to the IDS when fed with the same inputs:

$$\forall i \in \{1, \ldots, n\} \big(\Delta(o_i, a_i) < \epsilon\big) \tag{13}$$

that is:

$$\forall i \in \{1, \ldots, n\} \big(\Delta(DS(Q_{i-1}, q_i), IDS(Q_{i-1}, q_i)) < \epsilon\big) \tag{14}$$

where $\Delta(\cdot)$ measures the difference between two natural language sentences. In RBT, the difference is expected to be smaller than a threshold ϵ. Otherwise, a bug will be reported under the current test case. One dialogue test case might be capable of revealing multiple bugs in different rounds, which will be recognised as different bugs revealed by this particular test case.

The follow-up test case is defined as Q':

$$Q' = [q'_1, q'_2, \ldots, q'_{n'}] \tag{15}$$

where $n' \geq 2$, and n' may not be equal to n. Practically, Q' can be constructed from Q through perturbation operations, e.g., element permutation, element deletion, element addition, and compositions of these operations, that is:

$$\forall i \in \{1, \ldots, n'\} \exists j \in \{1, \ldots, n\} \big(q'_i = q_j \text{ and } q_j \text{ appears in } Q\big) \tag{16}$$

After perturbation, each q'_i in Q' may have different context when compared to their identical question q_j in Q. If

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

q'_i and q_j deliver the same answer, then we regard Q'_{i-1} and Q_{j-1} as equivalent context and denote them as $Q'_{i-1} \equiv Q_{j-1}$. With the equivalent context, it is expected that the output from dialogue system, o'_i, should be similarly close to the answer of IDS a_j when compared with o_j. That is, we have:

$$\forall i \in \{1, \ldots, n'\} \big(\Delta(o'_i, a_j) \approx \Delta(o_j, a_j)\big) \text{ iff } \big(Q'_{i-1} \equiv Q_{j-1}\big) \tag{17}$$

In practice, if a_j is inaccessible and verification of $Q'_{i-1} \equiv Q_{j-1}$ is feasible, we have the following:

$$\forall i \in \{1, \ldots, n'\} \big(\Delta(o'_i, o_j) \leq \epsilon\big) \text{ iff } \big(Q'_{i-1} \equiv Q_{j-1}\big) \tag{18}$$

where ϵ is the similarity thresholds. If the MR is violated in follow-up testing, a bug will be revealed using target question q_j and test seed D.

C. Dialogue-Level Perturbations in MORTAR

Unlike previous perturbations that operate at the single-turn [14], [15], [33], MORTAR employs dialogue-level perturbations. Existing perturbations in single-turn MT can only affect the target question utterance. In contrast, dialogue-level operations are more substantial, as they can alter the conversational context, thereby influencing the answerability of the target question in perturbed test cases. This shift enables the evaluation of language models in more complex, multi-turn settings. Although these modifications are broader in scope, they are still considered perturbations - systematic variations introduced to assess the quality of dialogue systems.

Given the original question sequence, $Q = [q_1, q_2, \ldots, q_n]$, the dialogue-level perturbation r is implemented with $P^r(\cdot)$ that produces a new sequence with n^r questions:

$$Q^r = P^r(Q) \tag{19}$$

$$Q^r = [q_1^r, q_2^r, \ldots, q_{n^r}^r]. \tag{20}$$

Let δ^r be the change of number of rounds after applying r:

$$\delta^r = |n^r - n|. \tag{21}$$

Each question $q_i^r, i \in \{1, \ldots, n^r\}$ in Q^r can be found in Q, but the round that q_i^r appears in Q^r might be different from the round in which the identical question q_j appears in Q, i.e. i might not equal to j, thus q_i^r might have an inequivalent context when compared to q_j. Such situations will need additional judgment to tell if the condition in Equation (17) is met. This is one of the key features of dialogue-level perturbations, whose effect is variable for each utterance in the test seed. However, this characteristic enables them to generate rarely-seen test cases for dialogue systems when compared with single-turn perturbations, and requires the system under test to adapt to the actual dialogue and output wisely, indicating higher potential effectiveness of testing.

In MORTAR, three fundamental and four derivative dialogue-level perturbations are proposed to produce new question sequences.

- **Perturbation RS**: round shuffle:

$$Q^{\text{RS}} = P^{\text{RS}}(Q) = [q_{i'} | i' = \varphi_{\text{RS}}(i), i = 1, \ldots, n] \tag{22}$$

where $\varphi_{\text{RS}}(i)$ is a bijective mapping from the original round index to the shuffled round index. All original questions are reordered and form the perturbed question sequence. Perturbation RS can be regarded as element permutation of Q.

- **Perturbation RR**: round reduction:

$$Q^{\text{RR}} = P^{\text{RR}}(Q) = [q_i | \mathbb{I}(i) = 1, i = 1, \ldots, n] \tag{23}$$

where $\mathbb{I} : \{1, \ldots, n\} \to \{0, 1\}$ is a randomly sampled mask function, $\sum_{i=1}^{n} \mathbb{I}(i) = n - \delta^{\text{RR}}$, and each element in Q^{RR} is from Q. That is, δ^{RR} questions are randomly removed from the original sequence, and the rest form the perturbed question sequence. Perturbation RR can be regarded as element deletion of Q.

- **Perturbation RD**: round duplication:

$$Q^{\text{RD}} = P^{\text{RD}}(Q) = \text{Insert}(Q, X), \tag{24}$$

where X is a random subset of Q with δ^{RD} elements, the function Insert$(\cdot)$ iteratively inserts element from X into a random position between two elements or the head or tail position, finally forms the perturbed question sequence Q^{RD} with $n + \delta^{\text{RD}}$ elements. In Q^{RD}, all original questions from Q are preserved in the original order with potential intervals, and each selected question in X appears one additional time at random positions. Perturbation RD can be regarded as element addition of Q.

Real-world multi-turn dialogues between users and dialogue systems are inherently noisy, and often contain topic shifts, repetitions, or contextual omissions [36], [37], which requires dialogue systems to possess strong robustness. Perturbation RS, RR and RD are three fundamental dialogue-level perturbations. They simulate complicated real-world usage of dialogue systems in diverse situations where users may pose questions in shifted, incomplete, or redundant contexts, forming the erratic conversational scenarios [38]. Specifically, the round shuffle (RS) perturbation models random conversational and abrupt topic shifting [39], [40]. The round reduction (RR) perturbation simulates context omission or the user's implicit assumption of prior knowledge [41], [42]. The round duplication (RD) perturbation reflects redundant querying or repeating for confirmation [43], [44]. It is necessary for testing approaches to simulate such noisy scenarios to stress test LLM-based dialogue systems, and failing to properly handle such scenarios reveals the incapability of the dialogue system. To further construct varied follow-up test cases, four derivative perturbations are employed, they are:

- **Perturbation RSR**: round shuffle and reduction:

$$Q^{\text{RSR}} = P^{\text{RSR}}(Q) = P^{\text{RR}}(P^{\text{RS}}(Q)). \tag{25}$$

Perturbation RSR is a combined operation of RS and RR. It shuffles the original question sequence then deletes δ^{RR} randomly chosen questions to form a perturbed question sequence.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

- **Perturbation RSD**: round shuffle and duplication:

$$Q^{\text{RSD}} = P^{\text{RSD}}(Q) = P^{\text{RD}}(P^{\text{RS}}(Q)). \tag{26}$$

Perturbation RSD is a combined operation of RS and RD, which shuffles then duplicates δ^{RD} randomly chosen questions, inserts the chosen questions into random positions to form a perturbed question sequence.

- **Perturbation RRD**: round reduction and duplication:

$$Q^{\text{RRD}} = P^{\text{RRD}}(Q) = P^{\text{RD}}(P^{\text{RR}}(Q)). \tag{27}$$

Perturbation RRD is a combined operation of RR and RD, which deletes δ^{RR} randomly chosen questions then duplicates δ^{RD} randomly chosen questions from the remaining questions.

- **Perturbation RSRD**: round shuffle, reduction and duplication:

$$Q^{\text{RSRD}} = P^{\text{RSRD}}(Q) = P^{\text{RD}}(P^{\text{RR}}(P^{\text{RS}}(Q))). \tag{28}$$

Perturbation RSRD is a combined operation of all three fundamental perturbations, which shuffles first, then deletes δ^{RR}, and finally duplicates δ^{RD} randomly chosen questions in the original question sequence.

To sum up, those dialogue perturbations simulate more complicated information-oriented real-world requests in dialogue, and test the performance of dialogue systems. They require dialogue systems to tell if sufficient information is given in context and perform different behaviours. These perturbations further extend the test coverage of dialogue systems with rarely seen situations. The examples of perturbed multi-turn test input sequences are provided in supplementary material.

D. MRs in MORTAR

For one specific question in the original question sequence, the context can be changed using dialogue-level perturbations. According to Eq. 17, if the perturbed context remains equivalent to the original context, i.e., the target question in perturbed context is asking the same fact as in the original context, the dialogue system is expected to respond with a semantically similar answer, regardless of whether the question is asked in the RBT or MT. Otherwise, for the same question with inequivalent contexts, the dialogue system is expected to respond with semantically different answers, as the questions are targeting different facts.

The equivalent context check $EC(\cdot)$ is a function that verifies whether the context of a question in a perturbed test case is equivalent to the context of the identical question in the original test case. The detailed implementation of $EC(\cdot)$ is introduced in Section III-E. Given the target question $q_i^r, i \in \{1, \ldots, n^r\}$ and its context Q_{i-1}^r in the perturbed test case, $EC(\cdot)$ compares whether Q_{i-1}^r is equivalent to context of the identical target question q_j in original test case Q, which is Q_{j-1}. If equivalent (noted as $Q_{i-1}^r \equiv Q_{j-1}$), $EC(\cdot)$ will give `True`, otherwise `False`, such that we have the following:

$$EC(Q_{i-1}^r, q_i^r) = \begin{cases} \text{True}, & \text{if } Q_{i-1}^r \equiv Q_{j-1} \\ \text{False}, & \text{Otherwise} \end{cases} \tag{29}$$

Given the test seed Q and a set of perturbation operations R, each r in R is one of the independently executed perturbations among RS, RR, RD, RSR, RSD, RRD, and RSRD, even the same perturbation in two executions may yield different test cases. For the original test input from Q_j and perturbed test input from Q_i^r, the dialogue system $DS(\cdot)$ respectively generates responses o_j and o_i^r:

$$DS(Q_{j-1}, q_j) \to o_j, \tag{30}$$

$$DS(Q_{i-1}^r, q_i^r) \to o_i^r, \tag{31}$$

where $i \in \{1, \ldots, n^r\}$, $j \in \{1, \ldots, n\}$, $q_i^r = q_j$, q_i^r is the target question in perturbed test case, q_j is the identical question in original test case, o_i^r and o_j are outputs in MT and RBT respectively, and Q_{i-1}^r and Q_{j-1} are contexts in MT and RBT respectively.

To measure the semantic similarity of two utterances, we use the natural language embedding model ($\text{Emb}(\cdot)$) to calculate the embedding vector of each utterance, then calculate the cosine similarity of the two embedding vectors. If the cosine similarity reaches the threshold ϵ, the two utterances will be regarded as semantically similar, otherwise semantically different.

Using equivalent context check function, MR1 and MR2 are defined as follows:

- **MR 1: Context-Preserving MR.** If a perturbed context Q_{i-1}^r is equivalent to Q_{j-1} in terms of supplementary information to the target question:

$$EC(Q_{i-1}^r, q_i^r) = EC(Q_{j-1}, q_j) = \text{True}, \tag{32}$$

the dialogue system is expected to produce an answer semantically similar to the original expected response:

$$\text{Expect: } \cos(\text{Emb}(o_i^r), \text{Emb}(a_j)) \geq \epsilon. \tag{33}$$

- **MR 2: Context-Altering MR.** If perturbed context Q_{i-1}^r of q_i^r lacks critical information to clarify q_i^r or is inequivalent to Q_{j-1}:

$$EC(Q_{i-1}^r, q_i^r) \neq EC(Q_{j-1}, q_j), \tag{34}$$

the dialogue system is expected to produce an answer semantically different from the original expected response:

$$\text{Expect: } \cos(\text{Emb}(o_i^r), \text{Emb}(a_j)) < \epsilon. \tag{35}$$

All perturbations can be fitted with MR1 or MR2. Given the context check results, it can be determined which MR and perturbation shall be matched for bug detection. In addition to individual MRs, we further propose group-based MRs that expect dialogue systems to adapt to and ensure consistent behaviour among different versions of context-changed questions.

For an original target question q_j from the original dataset, assuming there are multiple perturbed question sequences where the identical target question appears at different rounds with different contexts. We name the same question under different contexts as different *versions* of the original question q_j. After executing different perturbations in R, all different versions of q_j form a group q_j^R, and all outputs of the dialogue system under test form a group o_j^R:

$$q_j^R = \{q_i^r | q_i^r = q_j, r \in R\} \tag{36}$$

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

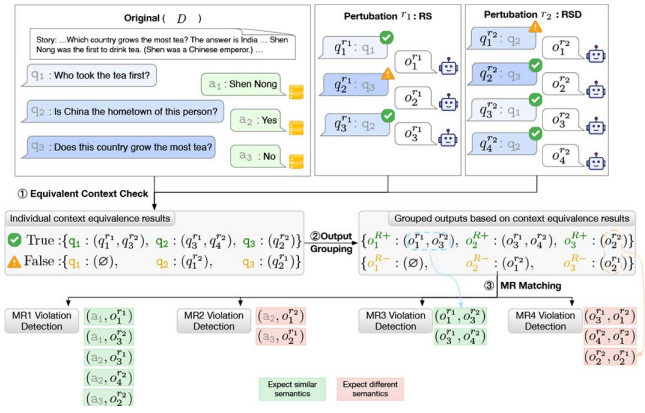


Fig. 3. Perturbation and MR violation detection. Given the perturbed test inputs and original test case, using the equivalent context check results, the outputs are matched with corresponding MR violation detection.

$$o_j^R = \{o_i^r | o_i^r = DS(Q_{i-1}^r, q_i^r), q_i^r = q_j, r \in R\} \tag{37}$$

The context of each q_i^r in corresponding test case may be equivalent or inequivalent to the original context of q_j in test seed. We split the outputs o_j^R into the output group of context-preserving perturbations of the original question, o_j^{R+}, and the output group of context altered perturbations, o_j^{R-}:

$$o_j^{R+} = \{o_i^r | EC(Q_{i-1}^r, q_i^r) = \text{True}, q_i^r \in q_j^R\}, \tag{38}$$

$$o_j^{R-} = \{o_i^r | EC(Q_{i-1}^r, q_i^r) = \text{False}, q_i^r \in q_j^R\}. \tag{39}$$

- **MR 3: Inner-Group Consistency MR.** Within group o_j^{R+}, different versions of q_j have equivalent contexts to the original, the dialogue system should not produce highly divergent answers, meaning the maximum difference within the group should be below threshold ϵ:

$$\text{Expect: } \min_{o_1, o_2 \in o_j^{R+}} \cos(\text{Emb}(o_1), \text{Emb}(o_2)) \geq \epsilon. \tag{40}$$

- **MR4: Inter-Group Divergence MR.** For two versions of q_j from the two groups, the system's outputs must be different and greater than ϵ:

$$\text{Expect: } \max_{o_1 \in o_j^{R+}, o_2 \in o_j^{R-}} \cos(\text{Emb}(o_1), \text{Emb}(o_2)) < \epsilon. \tag{41}$$

As shown in Fig. 3, given the original multi-turn dialogue ($D = [(q_1, a_1), (q_2, a_2), (q_3, a_3)]$) as a test seed, all questions can be regrouped as Q: $Q = [q_1, q_2, q_3]$. Two perturbations r_1 (RS) and r_2 (RSD) are adopted and generates two new question sequences: $Q^{r_1} = [q_1^{r_1}, q_2^{r_1}, q_3^{r_1}] = [q_1, q_3, q_2]$ and $Q^{r_2} = [q_1^{r_2}, q_2^{r_2}, q_3^{r_2}, q_4^{r_2}] = [q_2, q_3, q_1, q_2]$ respectively. The questions in Q may appear at different rounds in perturbed test cases, such that the identical question may have varied context and targeting information in perturbed test case when compared with the original test case. For example, q_1 is the first question in Q, and it is asked in the first round in Q^{r_1} as $q_1^{r_1}$ and the third round in Q^{r_2} as $q_3^{r_2}$.

The variance results in different contexts equivalence, some questions' contexts remain equivalent to the identical question in the original contexts while the others do not. For example, the context of $q_3^{r_2}$ in Q^{r_2} is $Q_2^{r_2} = [q_1^{r_2}, q_2^{r_2}] = [q_2, q_3]$, the identical question of $q_3^{r_2}$ in original test case Q is q_1, and the context of q_1 is Q_0 which is None. As q_1 is an independently answerable question without reliance on any context questions, the context of $q_3^{r_2}$ can be regarded as equivalent to the context of q_1, i.e., $EC(Q_2^{r_2}, q_3^{r_2}) = \text{True}$. Meanwhile, q_3 is the third question in Q, and it is asked in the second round in Q^{r_1} as $q_2^{r_1}$. Different from the aforementioned example, $EC(Q_1^{r_1}, q_2^{r_1}) = \text{False}$, where $Q_1^{r_1} = [q_1^{r_1}] = [q_1]$. Similarly for all questions in perturbed test cases, the equivalent context check (Step ① in Fig. 3) is conducted to obtain the individual context equivalence results. For example, q_2 is the second question in Q, it is also asked in the third round in Q^{r_1} as $q_3^{r_1}$, and in both the first and fourth round in Q^{r_2} as $q_1^{r_2}$ and $q_4^{r_2}$ respectively. The individual context equivalence results as following: $EC(Q_2^{r_1}, q_3^{r_1}) = \text{True}$, $EC(Q_0^{r_2}, q_1^{r_2}) = \text{False}$, and $EC(Q_3^{r_2}, q_4^{r_2}) = \text{True}$. The results are also presented in group for simplicity: $\text{True} : \{q_2 : (q_3^{r_1}, q_4^{r_2})\}$ and $\text{False} : \{q_2 : (q_1^{r_2})\}$.

Based on the individual context equivalence results, we group the outputs of system under test based on the questions' context equivalence to each corresponding question in Q (Step ② in Fig. 3). For example, q_1 is identical to $q_1^{r_1}$ and $q_3^{r_2}$, and $EC(Q_0^{r_1}, q_1^{r_1}) = EC(Q_2^{r_2}, q_3^{r_2}) = \text{True}$, such that the corresponding outputs $o_1^{r_1}$ and $o_3^{r_2}$ are grouped in o_1^{R+}, where $R = \{r_1, r_2\}$.

In step ③, using the grouped context equivalence results, MRs are adaptively matched and used to detect bugs. For example, MR1 is an individual MR and $o_3^{r_2}$ is an element of o_1^{R+}, according to the definition of MR1 in Eq. 32, we expect a_1 and $o_3^{r_2}$ to be semantically similar, as q_1 and $q_3^{r_2}$ are asking the identical information, although they have varied contexts. If a_1 and $o_3^{r_2}$ are semantically different, MR1 will be violated, indicating a bug is revealed in the dialogue system under test. Similarly for MR2, $o_2^{r_1}$ is an element of o_3^{R-}, according to the definition of MR2 in Eq. 34, we expect a_3 and $o_2^{r_1}$ to be semantically different. If a_3 and $o_2^{r_1}$ are semantically similar, MR2 will be violated and a bug will be revealed in the dialogue system under test. Violation to MR2 indicates the dialogue system not performing well in adapting to the context in multi-turn dialogues. For group-based MRs, take o_1^{R+} for example, two elements belong to this group: $o_1^{R+} : \{o_1^{r_1}, o_3^{r_2}\}$, according to the definition of MR3, we expect $o_1^{r_1}$ and $o_3^{r_2}$ to be semantically similar. And for MR4, o_3^{R+} has an element $o_2^{r_2}$ and o_3^{R-} has an element $o_2^{r_1}$, such that we expect $o_2^{r_2}$ and $o_2^{r_1}$ to be semantically different, as their questions are targeting different information in inequivalent contexts.

To our knowledge, the above formalisation is the first attempt to model the multi-turn MT for dialogue systems. It is worth noting that MORTAR's process is different from single-turn metamorphic testing methods. In existing methods, the perturbation or test seed and MR violation detection are coupled, such that the testing is inevitably semi-automated as manual selection or predefining the MR for violation detection is necessary. In multi-turn MT, the effects of dialogue-level perturbations are determined later with the equivalent context check process. This mechanism allows MORTAR to enhance the diversity of test

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

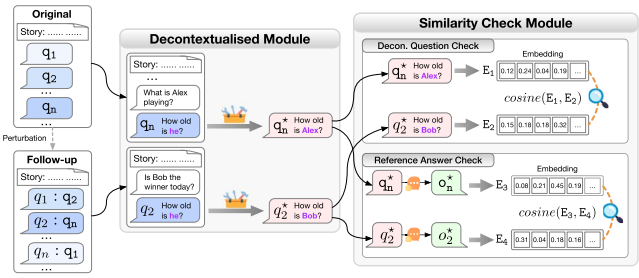


Fig. 4. Equivalent context check pipeline in MORTAR.

cases and maximise the usage of test seeds. Given that $EC(\cdot)$ is automated, the multi-turn metamorphic testing can be fully automated.

E. Equivalent Context Check

In Equation (29), the equivalent context check ($EC(\cdot)$) gives whether the context of a question in a perturbed test case is equivalent to the context in the original test case, and it is also a key component to judge whether an MR is applicable or not. As the effect of dialogue-level perturbations is dynamic, the answer expectation of some questions in perturbed test cases may differ from answers of the original question or the same question in other perturbations, such that different MRs shall be adopted according to the actual situation. To handle this issue and implement automated metamorphic testing, it is necessary to use the equivalent context check and dynamically match the appropriate MR according to the context of each question in the perturbed test case. To the best of our knowledge, there is no available approach in the literature that formalise or automate such a notion of contextual equivalence. As shown in Fig. 4, we propose an effective design of equivalent context check composed of 1) the decontextualisation, and 2) the similarity check.

1) Decontextualisation: Inspired by the question rewrite task in natural language processing [45], which extracts information from dialogue context and rewrite the target question into an independently answerable question, MORTAR adopts a prompt-based approach and use an LLM to rewrite and decontextualise the target questions with perturbed context. The prompt template is provided in supplementary material.

Given the context information, i.e., the reference article and the question sequence from context, a target question can be rewritten as a context-independent question that carries sufficient context on its own, while asking the same fact as the context-dependent form. For example, in Fig. 4, the target question "How old is he" is q_n in original test case and the q_2 in perturbed test case, and the perturbation can be RS or other derivative perturbations (see Section II-I-C). This target question cannot be answered independently unless the ellipsis information (who is "he") is clarified in the context.

In the original test case, the target question is decontextualised as "How old is Alex?". However, in the perturbed test case, it is decontextualised as "How old is Bob?" as the context of perturbed test case is **inequivalent** to the context of the original test case. Pronouns are common anaphoric expressions [46] in multi-turn dialogues that cause context reliance of target questions, while MORTAR is also capable of handling various types of ellipsis [35]. For example, verbal ellipsis involves the omission of a verb phrase that appeared earlier in the dialogue. If the context contains "Alex can speak French," and the target question is "Can Bob?", MORTAR identifies the missing predicate and rewrites the question as "Can Bob speak French?". If the contexts of the target question in test seed and the perturbed test case are equivalent, the decontextualised question will be similar in semantics and asking the same fact. Otherwise, the inequivalent contexts will result in different decontextualised questions, and the similarity check step will detect the difference to give equivalent context check result.

2) Similarity Check: The similarity check step compares whether two decontextualised questions are semantically similar, and the similarity indicates the equivalence of corresponding contexts. In MORTAR, similarity check is composed of two parts: 1) the decontextualised question check, and 2) the reference answer check.

In the question similarity check, the natural language embedding model is used to calculate the semantic embedding vectors of the decontextualised questions ($\mathtt{E}_1 = \mathrm{Emb}(\mathtt{q}_\mathtt{n}^*)$, $\mathtt{E}_2 = \mathrm{Emb}(q_2^*)$). The similarity of the decontextualised questions is given by the cosine similarity of the two embedding vectors: $Sim_Q = \cos(\mathtt{E}_1, \mathtt{E}_2)$.

In practice, the question decontextualisation occasionally rewrites questions with too much or irrelevant information. For example, the question "Where was he coming from?" is rewritten as "From where was Guy coming?" in the original test case, and "From where was Guy, who proposed to Virginia Duge, coming?" in the perturbed test case, since the perturbed test case has an additional round in context: "who proposed to Virginia Duge". Although they can be answered with the same answer, the similarity of the two decontextualised questions is not as high as other question pairs. Hence, merely relying on the question similarity check could mistakenly output "False" for such cases and result in increased false positive rates. To handle this issue, we add a reference answer check process. We use a prompt-based approach to obtain the reference answers of the two decontextualised questions ($\mathtt{o}_\mathtt{n}^*$ and o_2^*) with single-turn input, and compute the similarity of the embeddings ($\mathtt{E}_3 = \mathrm{Emb}(\mathtt{o}_\mathtt{n}^*)$ and $\mathtt{E}_4 = \mathrm{Emb}(o_2^*)$) of the reference answers of two decontextualised questions. The reference answers check does not generate test oracles, but used to filter out the noise in the output of question decontextualisation.

In this way, the overall similarity score is calculated by averaging the similarity of the decontextualised questions and the similarity of the reference answers. If the overall similarity score reaches a preset threshold ϵ_{EC}, the equivalent context check will indicate "True", otherwise "False":

$$EC(\cdot) = \begin{cases} \text{True}, & \text{if } \frac{\cos(\mathtt{E}_1,\mathtt{E}_2)+\cos(\mathtt{E}_3,\mathtt{E}_4)}{2} \geq \epsilon_{EC} \\ \text{False}, & \text{Otherwise} \end{cases} \tag{42}$$

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

To summarise, different questions in a perturbed test case may be matched with different MRs for violation detection, and this matching is driven by the automated equivalent context check ($EC(\cdot)$). Instead of delegating the equivalent/inequivalent verdict to an end-to-end LLM prompt, $EC(\cdot)$ in MORTAR limits the usage of LLM to constrained natural language processing tasks, and each step of $EC(\cdot)$ can therefore be independently verified, as reported in Section V.

IV. Design of Experiments

In this section, we introduce the research questions, setting of experiments, baseline methods, the dataset, dialogue systems under test, and relevant implementation details.

A. Research Questions

RQ 1: How does MORTAR perform in detecting bugs in LLM-based dialogue systems? We answer this research question by comparing the effectiveness and robustness of MORTAR with the baseline methods.

RQ 2: What is the quality of the bugs revealed by MORTAR? In this RQ, we analyse the characteristics and quality of detected bugs in terms of diversity, precision and uniqueness.

RQ 3: How effective are the components of MORTAR? In this RQ, we conduct an ablation study and investigate the contribution of the seven perturbations and four MRs to the overall performance, and the effectiveness of equivalent context check component.

RQ 4: How sensitive is MORTAR to its hyperparameter configurations? In this RQ, we examine the performance of MORTAR under different perturbation rate and semantic similarity threshold.

B. Data Preparation

Regarding the test dataset, we have the following expectations: *a*) focusing on fundamental capabilities of testing dialogue systems; *b*) have two human participants with one posing questions and another answering the questions; *c*) questions might have reliance on dialogue history; *d*) questions can be independently answered when given sufficient information so that the question can be decontextualised. CoQA [16] fundamentally meets all the above requirements. CoQA is a multi-turn reading comprehension dataset with topic in multiple domains. Each record is composed of a story and a multi-turn dialogue record. The multi-turn dialogue is produced by a human question-asker and a human answerer. It comprises a training set and a development set. We adopt the development set as test seeds. There are a total of 500 dialogues with 7983 questions.

C. Baselines

To the best of our knowledge, currently there are no multi-turn dialogue testing methods that provide a compatible and judge-free test oracle with explicit pass/fail criteria on fixed seed dialogues, hence no existing multi-turn baseline can be directly used for a fair performance comparison with MORTAR. Popular multi-turn evaluation benchmarks that rely on LLM judges yield graded or preference scores rather than MR-grounded pass/fail outcomes, and therefore do not constitute testing oracles. In contrast, single-turn metamorphic testing methods define automated, label-free oracles and are the closest feasible comparators. Therefore, we adapt two representative single-turn MT baselines: METAL [15] and QAAsker [13]. METAL is one of the latest and most effective MT frameworks for single-turn LLM testing. We use the four most effective single-turn MRs in METAL: the model's outputs on the original and the perturbed input should not differ, the perturbations are *synonym-replacement*, *add random word*, *introduce-typos*, and *convert-to-leet-format*. QAAsker is an effective and representative single-turn metamorphic testing method for QA systems. It constructs follow-up questions using three MRs with full test automation.

As the effect of single-turn MT may change the information content in each turn, we adopt their original implementation of perturbations on each input question and concatenate the perturbed question with original context to generate the follow-up test cases. This unified adaptation of single-turn metamorphic testing in multi-turn testing ensures the original single-turn MRs still strongly hold and the context remains fully informative to each perturbed question, such that they function as comparable baselines to MORTAR.

D. Test Objects

We select six open-source conversational LLMs and two representative closed-source LLMs and prompt each of them to function as a dialogue system (DS). In the prompts, the story is first provided as a reference, then the LLMs are required to answer questions with *concise and short* answer to reduce unrelated information, and they are required to answer "unknown" if they do not know the answer. When testing these dialogue systems, we simulate a user having multi-turn dialogues with the dialogue system. We feed the dialogue system with the question sequence in each metamorphic test case along with the dialogue history, then collect the generated answer to analyse response quality.

The selected LLMs for each dialogue system are presented in Table I. The open source models used are all instruction finetuned and their sizes range from 0.5 to 9 billion parameters, and the inference temperature is setup to be 0.2 and top-p parameter is set up to be 0.9 to reduce hallucination and repetitive degeneration while maintain relative deterministic behaviour in multi-turn dialogue [53], [54]. These dialogue systems can be regrouped into three sets: the first group is composed of DS1, DS2, and DS3. They are the same sourced models with different parameter sizes. The second group is composed of DS3, DS4, DS5 and DS6. They are all based on popular and recently published language models with similar model sizes. The third group is composed of DS7 and DS8 and they are based on representative closed-source LLMs GPT-5-nano and GPT-5. GPT-5 is one of the latest closed-source LLMs. GPT-5-nano is a fast and cost-effective version of GPT-5 and is more suitable for latency-sensitive environments where the quality of output shall not be compromised. In our experiment, the testing for

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE DIALOGUE SYSTEMS UNDER TEST

	Model	Type	Model Size
DS1	Qwen2-0.5B-Instruct [47]	Open-source	494M
DS2	Qwen2-1.5B-Instruct [47]	Open-source	1.54B
DS3	Qwen2-7B-Instruct [47]	Open-source	7.62B
DS4	Mistral-7B-Instruct-v0.3 [48]	Open-source	7.25B
DS5	Llama-3-8B-Instruct [49]	Open-source	8.03B
DS6	Gemma-2-9b-it [50]	Open-source	9.24B
DS7	GPT-5-nano [51]	Closed-source (Commercial)	Unknown
DS8	GPT-5 [52]	Closed-source (Commercial)	Unknown

open-source LLM-based dialogue systems is conducted locally. Using the instruction finetuned model weights, we implemented a series of dialogue systems powered by conversational LLMs with different sources and model sizes. The testing of closed-source models is conducted with API calls. We set the reasoning effort and output verbosity to low to ensure more deterministic behaviour and concise response.

E. Criteria Tailored to Multi-Turn Metamorphic Testing

When testing the dialogue system with multi-turn test cases, we use a series of multi-turn specific metrics to quantitatively describe the performance of a testing method, the metrics are summarised in Table II. They are adapted from existing single-turn testing metrics and extended to multi-turn testing to provide fine-grained quantitative performance measure of dialogue systems. Following the established naming convention in MT [14], [17], a *bug* is defined as a mistakenly answered question in RBT or a violation to MR in MT of the system under test, and a *detection* is the identification of those mistakes in a piece of output. A multi-turn test case is capable of revealing multi-turn bugs through verifying the output of each turn. If all questions are correctly answered and no MR is violated, the test case will be regarded as a *passed test case*. Otherwise, if one or more outputs are wrong or conflict the MR, the test case will be regarded as a *failed test case* (FTC), the turn with a bug will be regarded as a *failed turn*. The number of bugs per test case measures the density of bugs. The turn failure rate (Rate$^+$) is a general metric in testing, measuring the overall performance of dialogue systems and the efficiency of testing methods. In RBT, it indicates the ratio of questions that are not responded with expected answers. In metamorphic testing, it measures the ratio of MR violations, given by:

$$\text{Rate}^+ = \frac{\text{Total number of detected bugs}}{\text{Total number of detections}} \times 100\%.$$

As defined in Section III-B, a bug revealed using RBT is a question that the system under test responds with different answer from the test oracle, while a bug revealed using MT is a pair of outputs that violates MR. To delve into the type of revealed bugs, we categorise bugs based on how they are exposed when compared with RBT using the test seed.

An **L1-Bug** is a bug revealed by both the RBT and MT, i.e., a question is answered incorrectly in both RBT and MT. An **L2-Bug** is revealed when the RBT already reveals one or more bugs using the test seed, while other correctly answered questions

TABLE II
METRICS USED TO EVALUATE THE PERFORMANCE OF DIALOGUE SYSTEMS IN MULTI-TURN TESTING

Metrics	Explanation
Bugs	The number of bugs, a bug is an incorrect answer when compared with the ground truth answer or a violation with MR.
Bugs per test case (B/TC)	The number of bugs divided by the number of test cases.
Turn failure rate (Rate$^+$)	The number of detected bugs divided by the number of tested turns.
L1-Bugs	The number of L1-Bugs, an L1-Bug is revealed by the same question in both RBT and MT.
L2-Bugs	The number of L2-Bugs, an L2-Bug is revealed in MT using the correctly answered question from partially passed test case in RBT.
L3-Bugs	The number of L3-Bugs, an L3-Bug is revealed in MT using any question from fully passed test cases in RBT.

are incorrectly answered in MT. An L2-Bug is a new round-level bug. An **L3-Bug** is a bug revealed in the condition that, all questions from a test seed are answered correctly in RBT, while at least one question is incorrectly answered by the system under test in MT. An L3-Bug is a new dialogue-level bug, and can be regarded as a successful reuse of a fully-passed test seed from RBT.

Take q_3 in Fig. 3 for example, q_3 is the third question in RBT using the test seed and the second question ($q_2^{r_2}$) with perturbation r_2 in MT. If $o_2^{r_2}$ and a_3 violates MR1 ($EC(Q_1^{r_2}, q_2^{r_2}) = True$), it will be categorised as an 1) L1-Bug if: q_3 is incorrectly answered in RBT, or 2) L2-Bug if: q_3 is correctly answered in RBT while at least one other questions (can be q_1 or q_2) are incorrectly answered, or 3) L3-Bug if: all other questions in this test seed (q_1, q_2, and q_3) are all correctly answered in RBT.

The manual check will use F1 Score to assess the subjects, given by:

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where recall is given by:

$$\text{Recall} = \frac{\text{True positive}}{\text{True positive} + \text{False negative}}$$

and the precision is given by:

$$\text{Precision} = \frac{\text{True positive}}{\text{True positive} + \text{False positive}}.$$

Additionally, we use coefficient of variation (CV) to measure the spareness, calculated as:

$$CV = \frac{\sigma}{\mu},$$

where σ and μ are the standard deviation and mean.

F. Antecedent Testing: RBT With Original Dataset

1) Result of RBT: When conducting RBT on all dialogue systems under test with original test cases, all dialogue systems are exposed to the same test set of 7,983 questions from 500 original dialogue test cases. According to the result in Table III, substantial unexpected behaviour is observed. Among all dialogue systems, DS8 performs the best. The turn failure rate of DS8 is 86% lower than DS1. We generate follow-up test cases to further test the dialogue systems.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TEST RESULT OF RBT USING ORIGINAL TEST CASES

	DS1	DS2	DS3	DS4	DS5	DS6	DS7	DS8
Failed test cases	492	446	369	460	292	279	401	271
Ratio of failed test cases	98.40%	89.20%	73.80%	92.00%	58.40%	55.80%	80.20%	54.20%
Bugs	3,386	1,965	1,147	1,589	524	469	1,153	462
Rate$^+$	42.42%	24.61%	14.37%	19.90%	6.56%	5.87%	14.44%	5.79%

TABLE IV
THE F1 SCORES OF DIFFERENT MULTI-TURN ANSWER SIMILARITY THRESHOLD. HIGHER IS BETTER. $\epsilon = 0.6$ REACHES THE HIGHEST F1 SCORE

ϵ	0.4	0.5	0.6	0.7	0.8	0.9
F1 score	0.841	0.867	**0.874**	0.868	0.788	0.643

TABLE V
THE SUMMARY OF FOLLOW-UP TEST DATASET

	QAAsker	METAL	MORTAR
Follow-up test cases	1500	2,000	3,500
Maximum detectable bugs	9,796	24,803	57,010

2) Justification of answer similarity threshold (ϵ) in multi-turn testing scenarios: We conduct a manual check on RBT test results to justify the answer similarity threshold for multi-turn scenarios. The manual labelling is conducted by two authors. We randomly sample 20 responses of each dialogue system in RBT to form a total of 160 evaluated responses, and two authors manually label if the response is correct or not and give binary judgement. After independent labelling, the two authors discuss on the diverged judgements and create consensus labels. The manual labels are then used to verify the F1 score under different values of ϵ. According to the Table IV, $\epsilon = 0.6$ results in the highest F1 score, indicating best agreement with human judgements.

3) Generation of follow-up test cases: We use the original test dataset to generate the perturbed test cases with MORTAR and baseline methods to further test the dialogue systems. The follow-up testing datasets are summarised in Table V. Given the original dataset, all testing methods can generate enlarged follow-up test dataset. It is worth noting that the MORTAR-generated test cases cannot be used in RBT, as they only carry verifiable metamorphic relations rather than ground truth answers. The number of different categories of bug (L1 to L3, defined in Section IV-E) can also be interpreted as the comparison between MORTAR and RBT.

G. Experiment Environment and Hyper Parameter Settings

The local machine is equipped with 32-core processors, 64GB memory and an RTX 3090 GPU. The system runs Ubuntu 24.04 LTS and Python 3.9. Groq [55] is an LLM inference cloud service provider. The equivalent context check is implemented with the open-sourced gpt-oss-120B model accessed through Groq. This is the largest open-weight model freely accessible on Groq's production tier. We use the gte-large-en-v1.5 model [56] to compute the 1024-dimension embedding vectors. This embedding model achieved strong performance on English natural language embedding within the same model size category [57]. These configurations supports accurate equivalent context check and sentence embedding vector computation.

The semantic similarity threshold in equivalent context check ϵ_{EC} in Section III-E is set up to be 0.8 (justified in Section V-D), the answer similarity threshold ϵ in Section III-B is set to be 0.6 (justified in Section IV-F). By default, the perturbation ratios in RR, RD and derivative perturbations in Section III-C are all set to be 30%. We further analyse sensitivity to hyperparameters in RQ4.

TABLE VI
TESTING THE DIALOGUE SYSTEMS WITH MORTAR AND BASELINE METHODS. B/TC: NUMBER OF BUGS PER TEST CASE. RATE $^+$: TURN FAILURE RATE

DS	QAAsker			METAL			MORTAR		
	Bugs	B/TC	Rate$^+$	Bugs	B/TC	Rate$^+$	Bugs	B/TC	Rate$^+$
DS1	5,037	3.36	51.42%	12,940	6.47	52.17%	30,009	**8.57**	**52.64%**
DS2	3,399	2.27	34.70%	8,505	4.25	34.29%	22,166	**6.33**	**38.88%**
DS3	2,558	1.71	26.11%	6,093	3.05	24.57%	16,792	**4.80**	**29.45%**
DS4	2,414	1.61	24.64%	6,418	3.21	25.88%	16,784	**4.80**	**29.44%**
DS5	1,655	1.10	16.89%	3,773	1.89	15.21%	11,677	**3.34**	**20.48%**
DS6	1,727	1.15	17.63%	2,925	1.46	11.79%	11,262	**3.22**	**19.75%**
DS7	2,982	1.99	30.44%	7,553	3.78	**30.45%**	**16,409**	**4.69**	28.78%
DS8	1,865	1.24	19.04%	3,562	1.78	14.36%	11,773	**3.36**	**20.65%**
Mean	2,704	1.80	27.61%	6,471	3.24	26.09%	**17,109**	**4.89**	**30.01%**

V. RESULTS

In this section, we present the empirical results to answer RQs presented in Section IV-A, and discuss the information extraction, false positive detections, and the performance evaluation of different dialogue systems.

A. RQ1. Performance of MORTAR

To answer RQ1, we evaluate the overall performance of MORTAR and compare it to the baselines in revealing dialogue system bugs with follow-up testing in terms of effectiveness and robustness. The effectiveness is measured with the performance of revealing bugs in terms of number of bugs and turn failure rate. The efficiency is measured as the number of bugs revealed per test case. The robustness is measured by the performance variation when testing different dialogue systems.

1) Effectiveness: After testing the eight dialogue systems, the results are presented in Table VI. MORTAR is capable of detecting 532% more bugs than QAAsker and 164% more bugs than METAL, with 15.0% and 8.69% higher turn failure rate than METAL and QAAsker respectively. In general, MORTAR shows higher effectiveness than the baselines.

2) Efficiency: The metric bug per test case measures the bug density in testing. The higher this metric is, the more bugs the testing method can reveal through one test case execution, thus is more efficient. On average, MORTAR's test efficiency is 172% higher than QAAsker and 51% higher than METAL.

3) Robustness: When testing different dialogue systems, the turn failure rate (Rate$^+$) of QAAsker dropped by 33.37 percentage points (pp) from 51.42% (DS1) to 17.63% (DS6). METAL dropped by 40.38 pp from 52.17% (DS1) to 11.79% (DS6). MORTAR dropped by 32.89 pp from 52.64% (DS1)

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
THE COEFFICIENT OF VARIATION (CV) OF RATE$^+$ WHEN TESTING DIFFERENT DIALOGUE SYSTEMS. LOWER IS BETTER

	QAAsker	METAL	MORTAR
CV	0.389	0.475	**0.349**

TABLE VIII
THE MANUAL CHECK RESULTS OF PRECISION OF SAMPLED BUGS DETECTED BY DIFFERENT METHODS

	QAAsker	METAL	MORTAR
Precision	49.2%	60.8%	**75.8%**
Cohen's Kappa	0.716	0.683	0.733

to 19.75% (DS6), the performance drop is lower than that of the best baseline. We then measure the coefficient of variation (CV) of performance when testing different dialogue systems. In Table VII, the 10.3% lower CV than best-performed baseline QAAsker indicates better robustness of MORTAR in testing. Collectively, it is observed that MORTAR has better robustness than the baseline in testing different dialogue systems.

Answer to RQ1: MORTAR is 164% more effective, 51% more efficient, and 10.3% more robust than the single-turn MT baseline methods.

B. RQ2. Quality of Detected Bugs

To answer RQ2, we examine the quality of bugs in terms of their correctness, diversity and uniqueness.

1) Precision: To measure the precision of detected bug, we randomly sample 120 bugs detected by each testing method, such that a total of 360 bugs are manually checked. Two authors first independently give binary judgement of the correctness of each dialogue system output, then discuss uncertain cases and, where possible, reach a consensus on the final decision. A bug is classified as false positive if both evaluators agree that *a)* the input turn question is clear and understandable, and *b)* the dialogue system's output is factually correct, aligns with the answer requirements, and has only reasonably limited additional content if there is any. Otherwise, the bug will be classified as a true positive.

As described in Table VIII, MORTAR realises 24.6% higher precision in bug detection than the best-performing baseline METAL, with a higher Cohen's Kappa than baselines. This indicates a higher level of inter-rater agreement on MORTAR's detection outcomes. The lower Kappa score for METAL may reflect a greater degree of unresolved disagreement between the two evaluators. Further discussion of false positive detections is provided in Section V-E1. Overall, MORTAR demonstrates a higher proportion of true positive detections, indicating better precision compared to the baselines.

2) Diversity: Diversity indicates the testing method's capability of diversified usage of the original dataset to detect bugs. The definition of L1-Bugs, L2-Bugs and L3-Bugs are given in Section IV-E. According to the results in Table IX, on average, MORTAR reveals a significantly higher number of different types of bugs while keeping a balance of different types of bugs. The lowest coefficient of variation (CV) of the average number of bugs of different types indicates better diversity of the bugs detected by MORTAR.

TABLE IX
THE NUMBER OF DIFFERENT TYPES OF BUGS AND THE COEFFICIENT OF VARIATION (CV) OF AVERAGE NUMBER OF DIFFERENT TYPES OF BUG. THE LOWEST CV INDICATES BUGS REVEALED BY MORTAR ARE OF HIGHER DIVERSITY

DS	QAAsker			METAL			MORTAR		
	L1-Bugs	L2-Bugs	L3-Bugs	L1-Bugs	L2-Bugs	L3-Bugs	L1-Bugs	L2-Bugs	L3-Bugs
DS1	2,248	2,758	31	7,717	5,149	74	15,468	14,254	287
DS2	927	2,248	224	3,870	4,215	420	7,731	12,908	1,527
DS3	420	1,690	448	2,425	2,898	770	4,687	9,125	2,980
DS4	518	1,748	148	2,624	3,511	283	5,762	10,286	736
DS5	119	883	653	1,018	1,720	1,035	1,950	5,939	3,788
DS6	87	928	712	981	1,197	747	1,909	5,560	3,793
DS7	422	2,129	431	2,405	4,247	901	4,781	9,816	1,812
DS8	126	938	801	900	1,508	1,154	1,870	5,640	4,263
Mean	608	1,665	431	2,742	3,055	673	5,519	9,191	2,398
CV		0.604			0.490			**0.487**	

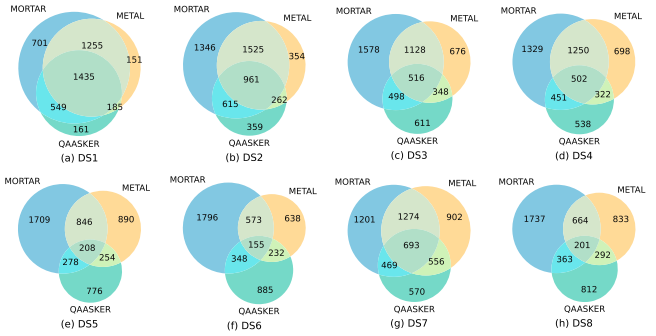


Fig. 5. Seed question overlap of detected bugs.

3) Uniqueness: To measure the uniqueness of bugs, we collect the seed question of bugs that are not revealed in RBT but by MORTAR and baseline methods. The seed question overlap is presented in Fig. 5. On average, 32.9% of the bugs revealed by QAAsker are unique, 26.9% of the bugs revealed by METAL are unique, while over 42.3% of the bugs revealed by MORTAR are unique, which is higher than all baselines.

Answer to RQ2: MORTAR is 24.6% more precise than baseline methods. The bugs revealed by MORTAR are more diverse, and over 42.3% of the revealed bugs are unique.

C. RQ3. Ablation Study

In MORTAR, seven dialogue-level perturbations are adopted to generate metamorphic test cases, and four MRs are used to form multi-turn metamorphic testing for dialogue systems. This RQ focuses on the contribution of each component to the overall effectiveness in test case generation and bug detection.

1) Ablation study on MR effectiveness: To measure the effectiveness of the MRs, we perform an ablation study by

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE X
THE NUMBER OF BUGS DETECTED BY MORTAR WHEN A CERTAIN MR IS EXCLUDED. FOR EXAMPLE, THE COLUMN MR1 INDICATES THE NUMBER OF DETECTED BUGS WITHOUT USING MR1 BUT ONLY USING MR2, MR3 AND MR4, AND THE PERCENTAGE IN THE BRACKET INDICATES THE DECREASE RATIO WHEN COMPARED WITH ORIGINAL PERFORMANCE USING THE FULL SET OF MRS. THE BOLD VALUES INDICATE THE MOST EFFECTIVE MR WHEN TESTING DIFFERENT DIALOGUE SYSTEMS

	Original	MR1	MR2	MR3	MR4
DS1	30,009	8,459 (**-71.81%**)	27,524 (-8.28%)	25,850 (-13.86%)	28,194 (-6.05%)
DS2	22,166	8,888 (**-59.90%**)	18,657 (-15.83%)	18,621 (-15.99%)	20,332 (-8.27%)
DS3	16,792	8,093 (**-51.80%**)	12,620 (-24.85%)	14,794 (-11.90%)	14,869 (-11.45%)
DS4	16,784	7,939 (**-52.70%**)	12,928 (-22.97%)	14,685 (-12.51%)	14,800 (-11.82%)
DS5	11,677	7,492 (-35.84%)	6,919 (**-40.75%**)	10,853 (-7.06%)	9,767 (-16.36%)
DS6	11,262	7,416 (-34.15%)	6,418 (**-43.01%**)	10,618 (-5.72%)	9,334 (-17.12%)
DS7	16,409	7,874 (**-52.01%**)	12,343 (-24.78%)	14,495 (-11.66%)	14,515 (-11.54%)
DS8	11,773	7,666 (-34.88%)	6,737 (**-42.78%**)	11,083 (-5.86%)	9,833 (-16.48%)
Mean	17,109	7,978 (**-53.37%**)	13,018 (-23.91%)	15,125 (-11.60%)	15,206 (-11.13%)

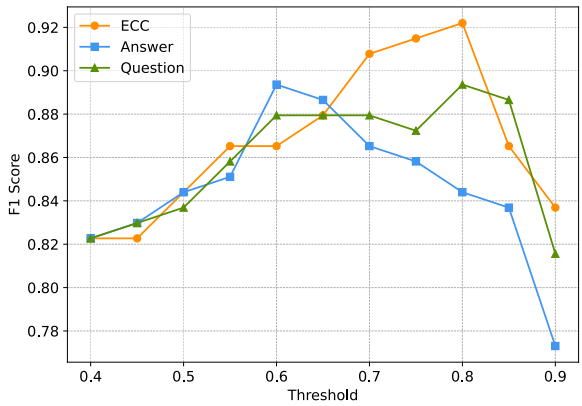


Fig. 6. The F1 score of equivalent context check results. Both decontextualised question similarity check and reference answer similarity check contribute to the overall effectiveness of equivalent context check.

collecting the number of detected bugs when each of the MRs is disabled one at a time. According to the result shown in Table X, among all MRs, MR1 generally contributes most to the overall effectiveness. MR2 shows unique major contribution when testing certain dialogue systems (e.g., DS5, DS6 and DS8). In general, all MRs contribute to the overall effectiveness, and the removal of any MR will cause an overall performance drop by over 11%.

2) Ablation study on perturbation effectiveness: To measure the effectiveness of perturbations, we perform an ablation study by collecting the number of detected bugs when each of the perturbations is disabled one at a time. According to the Table XI, perturbation RSD (dialogue round duplication and shuffle) contributes the most to overall effectiveness, with 19.55% of bugs revealing relies on it. The remaining perturbations also contribute significantly to the testing process. On average, the removal of any perturbation will cause at least 6.78% (column RR: dialogue round reduction) performance deterioration of MORTAR.

3) Ablation study on the equivalent context check: To measure the contribution of each component in equivalent context check, we randomly sample one dialogue from each perturbation, and a total of 141 rounds of questions are manually labelled with context equivalence results. The manual labels are compared against the performance of equivalent context without either decontextualised question similarity comparison or reference answer similarity comparison, and the results are presented in Fig. 6. Merely relying on decontextualised question similarity check or reference answer check cannot achieve the best performance. With both components, the overall F1 score reached 0.922, indicating high correctness of equivalent context check results.

Additionally, the equivalent context check can also be implemented with a pure LLM-as-a-judge approach without the explicitly predefined processing workflow. We compare the effectiveness of the equivalent context check in MORTAR and the LLM-as-a-judge approach with the identical LLM (gpt-oss-120B) on the manually labelled dataset. The prompt template used in the pure LLM judge-based equivalent context check is provided in the supplementary material. According to the prompt, the LLM is instructed to think step by step and judge whether the target question with different contexts shall be answered with the same answer. The LLM-as-a-Judge approach and $EC(\cdot)$ in MORTAR are given the same input, and the inference temperatures are both 0.5. According to the results shown in Table XII, $EC(\cdot)$ in MORTAR achieves a higher F1 score than the LLM-as-a-Judge approach. Within the sampled dataset, the LLM-as-a-Judge approach tends to give more judgement that the two given contexts are "equivalent" and less judgement of "inequivalent", resulting in high recall on the "equivalent" category while very low recall on the "inequivalent" category. As a comparison, $EC(\cdot)$ in MORTAR can identify such situations and give correct judgment of "inequivalent", indicating less bias and higher accuracy on the equivalent context check process. In general, $EC(\cdot)$ in MORTAR outperforms the pure LLM-as-a-judge baseline, the roles of LLM in $EC(\cdot)$ are constrained to executing specific natural language processing tasks within the explicitly predefined processing workflow, rather than making the final verdict directly, such that mitigates the bias of the LLM towards the "equivalent" category, which ensures the effectiveness of the equivalent context check.

To verify the robustness of equivalent context check under normal LLM variability, we repeat equivalent context check for 20 times on the manually labelled dataset and report how often the results flips between "equivalent" and "inequivalent". Following the practice in related research [58], the LLM inference temperature is setup to be 0.5. According to the experimental results, over 84% judgements given by equivalent context check are identical with the first attempt, and Fleiss' Kappa [59], [60] reach 0.63 (≥ 0.6: substantial agreement). These results indicate that while normal LLM variability does introduce minor fluctuations, the final binary decisions ("equivalent" or "inequivalent") made by equivalent context check demonstrate substantial agreement across multiple repeats. Therefore, the equivalent context check in MORTAR is regarded as robust under normal LLM variability.

It is worth noting that the bug counts reported in this section refer to MR violations rather than manually verified faults. As reported in Section V-B1 (Table VIII), 75.8% of MORTAR's

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
THE NUMBER OF BUGS DETECTED BY MORTAR WITHOUT EACH PERTURBATION. FOR EXAMPLE, THE COLUMN RS INDICATES THE NUMBER OF DETECTED BUGS WITHOUT USING PERTURBATION RS, AND THE PERCENTAGE IN THE BRACKET INDICATES THE DECREASE RATIO. THE **BOLD** VALUES INDICATE THE MOST EFFECTIVE PERTURBATION, AND THE *ITALIC* VALUES INDICATE THE LEAST EFFECTIVE PERTURBATIONS

	Original	RS	RR	RD	RSR	RSD	RRD	RSRD
DS1	30,009	25,392 (-15.39%)	27,978 (*-6.77%*)	24,618 (-17.96%)	27,297 (-9.04%)	24,127 (**-19.60%**)	27,047 (-9.87%)	26,517 (-11.64%)
DS2	22,166	18,740 (-15.46%)	20,660 (*-6.79%*)	18,180 (-17.98%)	20,196 (-8.89%)	18,003 (**-18.78%**)	19,912 (-10.17%)	19,628 (-11.45%)
DS3	16,792	14,207 (-15.39%)	15,640 (*-6.86%*)	13,794 (-17.85%)	15,289 (-8.95%)	13,504 (**-19.58%**)	15,112 (-10.00%)	14,789 (-11.93%)
DS4	16,784	14,018 (-16.48%)	15,653 (*-6.74%*)	13,978 (-16.72%)	15,142 (-9.78%)	13,596 (**-18.99%**)	15,223 (-9.30%)	14,755 (-12.09%)
DS5	11,677	9,757 (-16.44%)	10,901 (*-6.65%*)	9,754 (-16.47%)	10,515 (-9.95%)	9,344 (**-19.98%**)	10,552 (-9.63%)	10,275 (-12.01%)
DS6	11,262	9,388 (-16.64%)	10,483 (*-6.92%*)	9,436 (-16.21%)	10,147 (-9.90%)	8,995 (**-20.13%**)	10,223 (-9.23%)	9,877 (-12.30%)
DS7	16,409	13,802 (-15.89%)	15,311 (*-6.69%*)	13,594 (-17.16%)	14,812 (-9.73%)	13,129 (**-19.99%**)	14,867 (-9.40%)	14,500 (-11.63%)
DS8	11,773	9,820 (-16.59%)	10,968 (*-6.84%*)	9,862 (-16.23%)	10,634 (-9.67%)	9,413 (**-20.05%**)	10,647 (-9.56%)	10,325 (-12.30%)
Mean	17,109	14,390 (-15.89%)	15,949 (*-6.78%*)	14,152 (-17.28%)	15,504 (-9.38%)	13,764 (**-19.55%**)	15,448 (-9.71%)	15,083 (-11.84%)

TABLE XII
PERFORMANCE COMPARISON BETWEEN EQUIVALENT CONTEXT CHECK ($EC(\cdot)$) IN MORTAR AND THE APPROACH BASED ON LLM-AS-A-JUDGE. THE PRECISION AND RECALL ARE CALCULATED ON BOTH MANUAL LABELS OF EQUIVALENT AND INEQUIVALENT RESPECTIVELY, THE MICRO-F1 SCORE (F1 SCORE) INDICATES THE OVERALL CORRECTNESS IN EQUIVALENT CONTEXT CHECK. THE BOLD VALUES INDICATES BETTER PERFORMANCE

	Equivalent		Inequivalent		F1 Score
	Precision	Recall	Precision	Recall	
$EC(\cdot)$ in MORTAR	**0.949**	0.957	**0.792**	**0.760**	**0.922**
LLM-as-a-Judge	0.827	**0.991**	0.500	0.040	0.823

detections are confirmed as genuine faults by manual inspection. The ablation results above can be scaled by this factor to obtain conservative estimates of true bug counts.

> **Answer to RQ3:** All perturbations and MRs contribute to the effectiveness of test case generation and bug discovery. Every perturbation and MR accounts for more than 6.78% and 11.13% to the overall bug detection performance respectively. Each component in equivalent context check contributes to the overall performance, and the equivalent context check is robust under normal LLM variability.

D. RQ4: Sensitivity Analysis

In this RQ, we analyse the sensitivity of MORTAR to different perturbation ratio and semantic similarity thresholds.

1) Sensitivity to perturbation ratio: To analyse the sensitivity to the perturbation ratio, we chose DS5 as the subject as it shows medium performance. Perturbation ratios ranging from 0.1 to 0.5 with a step of 0.2 were included for comparison. According to the results shown in Fig. 7, no significant performance variation of MORTAR was observed. As the perturbation ratio rises, the number of perturbed questions and bugs show a minor decrease, but the turn failure rate (Rate$^+$) is insensitive to different perturbation ratios. Moreover, the distribution of different types of bugs and unique bugs shows no major changes. It can be concluded that MORTAR's performance is robust in the tested perturbation ratio range.

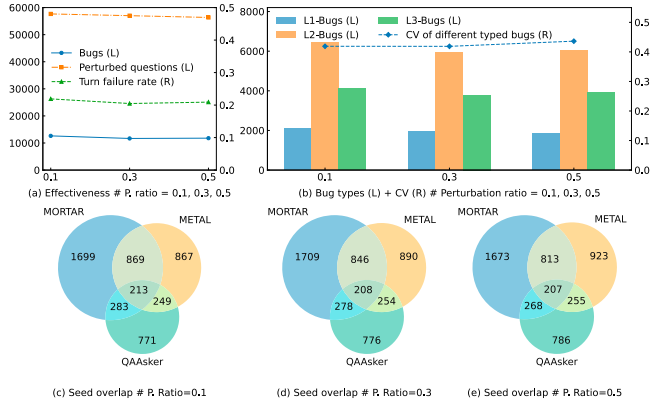


Fig. 7. Sensitivity to perturbation ratio.

2) Sensitivity to equivalent context check threshold: The sensitivity analysis result of different equivalent context check threshold is shown in Fig. 6. It is observed that with overall threshold 0.8, equivalent context check reaches its best measured performance. This corresponds with the design of equivalent context check. Some questions might contain redundant information and caused the similarity to be lower than other pairs. Reference answer check functions as an auxiliary check to allow those cases to be correctly judged. MORTAR is sensitive to equivalent context check threshold, the optimal performance is observed at threshold 0.8.

> **Answer to RQ4:** MORTAR is insensitive to perturbation ratio in selected value range, but sensitive to the equivalent context check threshold where 0.8 is determined as the optimal value.

E. Discussion

1) Case study: the false positive detections: The most common false positive detections are overlong answers. For example, the short answer "Yes" is expected in yes-no questions, the prompt also requests the answer to be concise, exact and short. As some questions might require detailed analysis, the DS tend to explain the answer with overlong paragraphs, as a result, the semantic similarity is lower than the threshold. The labellers

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

report that some reasonably wordy outputs of dialogue systems could be regarded as false positive detections, e.g. the expected answer: "In their home" and the output: "Mike McLelland and his wife Cynthia were found in their home in Kaufman County, Texas." do not have high semantic similarity (0.56), but the output is actually acceptable. Nevertheless, considerable outputs exceed the necessary explanation and give exceedingly long paragraphs, they are regarded as true positives. We expect future improvement in semantic similarity measurement to reduce false positive rates.

Besides, during manual check, both participants reported that METAL's perturbation brings too much noise to the original question, and considerable questions' semantics have been altered, which contradicts the expectation. For example, in multi-turn testing with the theme of biology, a question *"What is a gene?* is perturbed with *add random word* perturbation in METAL, and the perturbed question *"What is Pear a gene?"* significantly changed the semantics of the sentence. The dialogue system uses its internal knowledge to answers this question with explanations about the genes in fruits. Besides, the synonym replacement cannot guarantee robust behaviours. For example, the question "Who is in charge?" is perturbed into "World-Health-Organisation is in charge?" where "Who" is replaced by "World-Health-Organisation" which is improper.

The common issue with QAAsker is the quality of generated test cases. The rule-based sentence analysis in QAAsker cannot cover situations beyond the predefined rules, especially when ellipsis phenomenon is frequently observed in multi-turn dialogue. For example, the question "What was he doing before his death?" with the original answer "returning from a convenience store." is reformed with QAAsker MR1 into another wh-question "When was he returning from a convenience store?" and expects the answer "his death". DS8 answers the perturbed question with "February 26, 2012" which is verifiable in reference text and shows strong reasoning capability, this bug is regarded as a typical false positive.

2) Case study: the root cause of bugs: The categorisation of failures is based on the participants' reported observations in manual check in Section V-B1. The common types of failures are: a) hallucinated content, b) failed prompt instruction following, c) imperfect context reasoning and d) mistakenly triggered guardrail. The hallucinated content is the situation that the DS outputs content with information never mentioned in the reference document and context. For example, the question "Where did Cotton's mother put Cotton to clean the paint off?" should be answered with "a bucket of water", while DS1 outputs "It's possible for Cotton's mommy possibly rubbed her face on cotton to clean it off." which cannot be verified by the reference story. For failed prompt instruction following, all DS are required to provide "concise and short" answer without explanation, while from DS1 to DS8 all occasionally provide unnecessary analysis or "Explanation" tagged content. The imperfect context reasoning is the phenomenon that the DS cannot correct the misunderstanding in context and follows a wrong dialogue trajectory. It is more common for DS1 and DS2 whose performance are relatively weaker in testing. The mistakenly triggered guardrail is only observed with DS7. In the whole testing, 27 questions are rejected by DS7 as they are flagged as potentially violating the usage policy, while same questions are successfully answered by DS8. We anticipate the root cause to be ill-suited guardrail of GPT-5-nano model.

3) On the performance evaluation of dialogue systems: As depicted in Section IV-D, our experiments are conducted on eight dialogue systems that are based on different LLMs in terms of source and parameter size. DS1, DS2 and DS3 are based on LLMs of the same source but different parameter sizes, they form the comparison group 1. DS3, DS4, DS5 and DS6 are based on different sourced models with similar parameter sizes (7-9 billion), they form the comparison group 2. DS7 and DS8 are based on closed-source LLMs and form the comparison group 3. In group 1 and group 3, a scaling trend is observed through the trend of Rate^+. Larger models exhibit fewer bugs and a lower Rate^+. In group 2, the performance of different DS is at a similar level. To compare between open-sourced LLMs and closed-source LLMs, the GPT-5 LLM that DS8 is based on, is anticipated with larger parameter size than the tested open-source LLMs. GPT-5 model performs well on many benchmarks [6], [61], while it is not performing significantly better than other DS in multi-turn MT. Larger language models are typically capable of strong memory of knowledge in the training dataset and better reasoning capability [62]. However, according to the test results in Table X, LLM with a larger parameter size exhibits higher proportions of violations to MR2 and MR4. This contradicts the trend of increased reasoning capability as expectations. While the exact architectural limitations of GPT-5 model remain unclear, the results indicate that highly scaled models may still struggle with complex contextual perturbations. MORTAR is capable of reusing this dataset and building multi-turn metamorphic testing to reveal new bugs. Additionally, we expect the integration of MORTAR in model training to be beneficial to avoid over-fitting and improve the generalisation in real-world applications.

4) On the mitigation method: A potential method that enhances the capability of LLM-based dialogue systems is to increase the reasoning effort of backbone LLMs [63]. We configure the reasoning effort of the GPT-5-nano model to high and form DS9, all other settings are kept the same as DS7. DS9 is tested with MORTAR using default parameters in RQ1. According to the results shown in Table XIII, when configured with a higher reasoning effort of the backbone LLM, the number of bugs is decreased by 12.76%. Meanwhile, the number of output tokens is increased by over seven times. The additional output tokens form the reasoning content, resulting in fewer bugs in testing. Increasing the reasoning effort of the backbone LLM of the dialogue system appears to be an effective mitigation method. Besides, the detected bugs are potentially helpful to improve the performance of LLM-based dialogue systems. The bugs can also be regarded as negative samples, and using the outputs that violates MRs (e.g., MR1 and MR2) as negative preferences, post-training approaches [64], [65] can finetune the LLM to learn from the mistakes and realise better performance. We expect this future work to explore this direction and improve LLM-based dialogue systems with bugs revealed by MORTAR.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
THE PERFORMANCE OF HIGH REASONING EFFORT (DS9) AND LOW REASONING EFFORT (DS7) WHEN TESTED WITH MORTAR

	Bugs	B/TC	Rate$^+$	Output tokens
DS7	16,409	4.69	28.78%	4.59M
DS9	14,316	4.56	25.11%	37.99M

VI. THREATS TO VALIDITY

1) Perturbation and equivalent context check: The proposed dialogue-level perturbations rely on randomness to generate follow-up test cases, which cannot gain fine-grained control over the actual effect on follow-up test cases without scenario-specific designs. Therefore, an equivalent context process check is proposed to judge the actual effect of perturbation and guide the selection of proper MR. The framework of MORTAR enables fully automated metamorphic testing, while it might introduce wrong judgments and cause false positive detections. As there does not exist any plug-and-play tool that is primarily designed for the purpose of equivalent context checking, we crafted a feasible checking process to automate MORTAR. We further conduct a sensitivity analysis of the threshold and an ablation study in the equivalent context check to verify the framework design, and manually check the bugs to minimise the threats to validity.

2) Generalisability of results: This study assesses MORTAR using the multi-domain multi-turn QA dataset and eight LLMs against single-turn MT baselines. The dataset selection is aligned with the scope of this study, while taxonomies of dialogue in both linguistics and natural language processing are much broader in scope and sophisticated in category. When using different task-specific and domain-specific seed datasets, e.g., code generation, or testing LLMs not included in this study, the effectiveness of MORTAR may vary. Although we aim to include diverse models and scenarios, the results might not fully generalise to other domains. We endeavour to extend MORTAR to a broader scope in future work.

VII. RELATED WORK

A. Testing LLM-Based Dialogue Systems

Current dialogue test cases are generally from human-generated datasets and LLM-generated datasets. SQuAD and SQuAD 2.0 are classic single-turn QA datasets that have been widely used for training and testing of QA systems [66], [67]. CoQA dataset is a free-form multi-turn conversational QA dataset with unanswerable questions [16]. MuTual is a multi-turn reasoning-based dialogue dataset in open-domain [68]. MT-Bench is a 2-turn open-end dialogue dataset that is used to evaluate chatbots' conversation performance [10]. The "LLM-as-a-judge" approach is reported as feasible when comparing LLMs' alignment with human preference, and using LLM to generate test cases became a recent trend [9], [25], [26], [27], [28]. However, it is worth noting that the "LLM-as-a-judge" approach comes with intrinsic shortcomings [10], [69], and the evaluations are restricted to LLM judge scores but not pass-fail criteria. Besides, training data contamination is receiving increasing attention [24]. The area calls for more complete and reliable testing for LLM-based dialogue systems.

B. Metamorphic Testing

To alleviate the oracle problem in software testing, MT was first introduced in 1998 [12]. Later and up to now, MT is commonly used in both traditional software systems [70], [71] and machine learning systems [13], [72], [73]. The advantages of MT are simplicity in concept, the straightforward implementation, ease of automation, and low cost [31]. These lead to the potential of effective testing for complicated systems, e.g., the compiler [74] the black-boxed GenAI systems [4]. Specifically, METAL uses utterance perturbations and MR templates to evaluate LLMs [15]. Ontology-based MT provides concrete task-oriented test cases but falls short in generalisation [75]. KGIT uses knowledge graphs as test seeds and builds a metamorphic testing-based scheme to implement large-scale inference tests [17]. Drowzee uses metamorphic testing and constrained logic programming to detect hallucinations in LLMs [21]. DialTest uses a set of MRs to effectively generate test cases for dialogue systems [33]. Existing MT approaches lack systematic analysis and usage of dialogue-level perturbations and multi-turn dialogue test case generation. Multi-turn dialogue testing remains challenging.

VIII. CONCLUSION AND FUTURE WORKS

LLM-based dialogue systems are now widely used in real-world applications. While there exist many single-turn testing methods, they are limited to the single-turn scenarios, leaving the real-world multi-turn usage scenario significantly under-explored. In this research, a multi-turn metamorphic testing approach, MORTAR, is proposed to mitigate the oracle problem in dialogue system testing. In MORTAR, a series of formalised MRs and dialogue-level perturbations are proposed and implemented with the multi-turn MT framework to generate follow-up test cases and reveal bugs in LLM-based dialogue systems. Given an automated MR matching mechanism, MORTAR can be fully automated. The test results show MORTAR is more effective than the most effective single-turn metamorphic testing baseline. MORTAR detects over 164% more bugs than the single-turn MT baseline, and over 42% of them are unique bugs. The revealed bugs are of higher quality when compared with the baseline in terms of diversity, precision and uniqueness. The component contribution analysis further validates the effectiveness of components in MORTAR.

We wish to point out that *(i)* these four types of MRs are not restricted to the seven types of perturbations introduced in this paper, and *(ii)* different application domains for the multi-turn dialogue systems may have different MRs as their characterisations. MORTAR is expected to inspire both dialogue system developers and researchers to build more comprehensive testing methods for dialogue systems. In the future, we intend to refine the equivalent context check process and reduce the false positive detections. The semantic similarity measurement can

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

also be enhanced to improve the accuracy of judgment. Besides, the detected bugs can be further used as negative training sets, and pave the way for improving LLM-based dialogue systems in future work.

REFERENCES

[1] "ChatGPT." OpenAI. Accessed: Oct. 10, 2025. [Online]. Available: https://chat.openai.com/

[2] "Ollama documentation." Ollama. Accessed: Oct. 10, 2025. [Online]. Available: https://docs.ollama.com/

[3] Y. Chang et al., "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.

[4] A. Aleti, "Software testing of generative AI systems: Challenges and opportunities," in *Proc. IEEE/ACM Int. Conf. Softw. Eng., Future Softw. Eng. (ICSE-FoSE)*, 2023, pp. 4–14.

[5] Z. Yang et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2369–2380.

[6] D. Hendrycks et al., "Measuring massive multitask language understanding," 2020, *arXiv:2009.03300*.

[7] ShareGPT, "ShareGPT Data," 2023. Accessed: Oct. 29, 2024. Online]. Available: https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

[8] N. Li et al., "LLM defenses are not robust to multi-turn human jailbreaks yet," 2024, *arXiv:2408.15221*.

[9] G. Bai et al., "MT-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 7421–7454. [Online]. Available: https://aclanthology.org/2024.acl-long.401

[10] L. Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 46595–46623.

[11] H. Huang et al., "On the limitations of fine-tuned judge models for LLM evaluation," 2024. [Online]. Available: https://arxiv.org/abs/2403.02839

[12] F. Chan, T. Y. Chen, S. C. Cheung, M. F. Lau, and S.-M. Yiu, "Application of metamorphic testing in numerical analysis," in *Proc. IASTED Int. Conf. Softw. Eng. (SE),* 1998, pp. 191–197.

[13] S. Chen, S. Jin, and X. Xie, "Testing your question answering software via asking recursively," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 104–116.

[14] Q. Shen, J. Chen, J. M. Zhang, H. Wang, S. Liu, and M. Tian, "Natural test generation for precise testing of question answering software," in *Proc. 37th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2022, pp. 1–12.

[15] S. Hyun, M. Guo, and M. A. Babar, "Metal: Metamorphic testing framework for analyzing large-language model qualities," in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 117–128.

[16] S. Reddy, D. Chen, and C. D. Manning, "CoQA: A conversational question answering challenge," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 249–266, Apr. 2019.

[17] J. Wang, Y. Li, Z. Chen, L. Chen, X. Zhang, and Y. Zhou, "Knowledge graph driven inference testing for question answering software," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[18] Y. Fan and X. Luo, "A survey of dialogue system evaluation," in *Proc. IEEE 32nd Int. Conf. Tools Artif. Intell. (ICTAI)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1202–1209.

[19] Z. Guo et al., "Evaluating large language models: A comprehensive survey," 2023, *arXiv:2310.19736*.

[20] S. Mehri, M. Eric, and D. Hakkani-Tur, "DialoGLUE: A natural language understanding benchmark for task-oriented dialogue," 2020, *arXiv:2009.13570*.

[21] N. Li, Y. Li, Y. Liu, L. Shi, K. Wang, and H. Wang, "Drowzee: Metamorphic testing for fact-conflicting hallucination detection in large language models," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024, pp. 1843–1872, doi: 10.1145/3689776.

[22] Z. Li, X. Xu, T. Shen, C. Xu, J.-C. Xu, Y. Lai, C. Tao, and S. Ma, "Leveraging large language models for nlg evaluation: Advances and challenges," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Assoc. for Comput. Linguistics, Nov. 2024, pp. 16028–16045. [Online]. Available: https://aclanthology.org/2024.emnlp-main.896/

[23] J. Ou et al., "DialogBench: Evaluating LLMs as human-like dialogue systems," 2023, *arXiv:2311.01677*.

[24] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," 2024, *arXiv:2410.05229*.

[25] Y. Sun et al., "Parrot: Enhancing multi-turn instruction following for large language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 9729–9750.

[26] W.-C. Kwan et al., "MT-Eval: A multi-turn capabilities evaluation benchmark for large language models," 2024, *arXiv:2401.16745*.

[27] X. Wang et al., "MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback," 2023, *arXiv:2309.10691*.

[28] H. Duan et al., "BotChat: Evaluating LLMs' capabilities of having multi-turn dialogues," in *Findings Assoc. Comput. Linguistics: NAACL*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 3184–3200. [Online]. Available: https://aclanthology.org/2024.findings-naacl.201

[29] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 805–824, Sep. 2016.

[30] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," 2020, *arXiv:2002.12543*.

[31] T. Y. Chen et al., "Metamorphic testing: A review of challenges and opportunities," *ACM Comput. Surv. (CSUR)*, vol. 51, no. 1, pp. 1–27, 2018.

[32] J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine learning testing: Survey, landscapes and horizons," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 1–36, Jan. 2022.

[33] Z. Liu, Y. Feng, and Z. Chen, "DialTest: Automated testing for recurrent-neural-network-driven dialogue systems," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2021, pp. 115–126.

[34] K. Tu, M. Jiang, and Z. Ding, "A metamorphic testing approach for assessing question answering systems," *Mathematics*, vol. 9, no. 7, 2021, Art. no. 726.

[35] X. Zhang, C. Li, D. Yu, S. Davidson, and Z. Yu, "Filling conversation ellipsis for better social dialog understanding," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 05, pp. 9587–9595.

[36] F. Mo et al., "A survey of conversational search," *ACM Trans. Inf. Syst.*, vol. 43, no. 6, pp. 1–50, 2025.

[37] F. Shi et al., "Large language models can be easily distracted by irrelevant context," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 31210–31227.

[38] D. R. Traum, *A Computational Theory of Grounding in Natural Language Conversation.* New York: Univ. of Rochester, 1995.

[39] J. Isaacs and K. Rawlins, "Conditional questions," *J. Semantics*, vol. 25, no. 3, pp. 269–319, Aug. 2008, doi: 10.1093/jos/ffn003.

[40] Y. Hwang, Y. Kim, Y. Jang, J. Bang, H. Bae, and K. Jung, "MP2D: An automated topic shift dialogue generation framework leveraging knowledge graphs," in *Proc. Conf. Empirical Methods in Natural Lang. Process.*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, FL, USA: Assoc. for Comput. Linguistics, Nov. 2024, pp. 17682–17702. [Online]. Available: https://aclanthology.org/2024.emnlp-main.979/

[41] H. H. Clark and S. E. Brennan, "Grounding in communication," Perspectives on Socially Shared Cognition, L. B. Resnick, J. M. Levine, & S. D. Teasley (Eds.). American Psychological Association, 1991, pp. 127–149. https://doi.org/10.1037/10096-006.

[42] Z. Pan, K. Bai, Y. Wang, L. Zhou, and X. Liu, "Improving open-domain dialogue systems via multi-turn incomplete utterance restoration," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 1824–1833.

[43] M. J. Pickering and S. Garrod, "Toward a mechanistic psychology of dialogue," *Behav. Brain Sci.*, vol. 27, no. 2, pp. 169–190, 2004.

[44] Y. Jang, K.-i. Lee, H. Bae, H. Lee, and K. Jung, "IterCQR: Iterative conversational query reformulation with retrieval guidance," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico, Association for Computational Linguistics, Jun. 2024, pp. 8121–8138. [Online]. Available: https://aclanthology.org/2024.naacl-long.449/

[45] A. Elgohary, D. Peskov, and J. Boyd-Graber, "Can you unpack that? Learning to rewrite questions-in-context," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 5918–5924.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.

[46] R. Sukthanker, S. Poria, E. Cambria, and R. Thirunavukarasu, "Anaphora and coreference resolution: A review," *Inf. Fusion*, vol. 59, pp. 139–162, Jul. 2020.
[47] A. Yang et al., "Qwen2 technical report," 2024, *arXiv:2407.10671*.
[48] Mistral, "Mistral-7b-instruct-v0.3 model," 2024. [Online]. Available: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3
[49] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
[50] G. Team, "Gemma," Kaggle, 2024. [Online]. Available: https://www.kaggle.com/m/3301
[51] "GPT-5 Nano (model documentation)." OpenAI. Accessed: Oct. 5, 2025. [Online]. Available: https://platform.openai.com/docs/models/gpt-5-nano
[52] "GPT-5 (model documentation)." OpenAI. Accessed: Oct. 5, 2025. [Online]. Available: https://platform.openai.com/docs/models/gpt-5
[53] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," 2019, *arXiv:1904.09751*.
[54] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
[55] "Groq is fast AI interface." Groq. Accessed: Oct. 5, 2025. [Online]. Available: https://groq.com/
[56] X. Zhang et al., "mGTE: Generalized long-context text representation and reranking models for multilingual text retrieval," 2024, *arXiv:2407.19669*.
[57] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," 2022, *arXiv:2210.07316*.
[58] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *Proc. 11th Int. Conf. Learn. Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=1PL1NIMMrw
[59] R. Vashurin et al., "Benchmarking uncertainty quantification methods for large language models with LM-polygraph," *Trans. Assoc. Comput. Linguistics*, vol. 13, pp. 220–248, 2025.
[60] J. Correia et al., "A Comparison of conversational models and humans in answering technical questions: The firefox case," in *Proc. 48th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2026.
[61] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub Issues?" 2023, *arXiv:2310.06770*.
[62] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.
[63] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 24824–24837.
[64] X. Mao et al., "As simple as fine-tuning: LLM alignment via bidirectional negative feedback loss," in *Proc. 13th Int. Conf. Learn. Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=fsX9nFwMNj
[65] R. Zhang, L. Lin, Y. Bai, and S. Mei, "Negative preference optimization: From catastrophic collapse to effective unlearning," in *Proc. 1st Conf. Lang. Modeling*, 2024. [Online]. Available https://openreview.net/forum?id=MXLBXjQkmb
[66] P. Rajpurkar, "Squad: 100,000+ questions for machine comprehension of text," 2016, *arXiv:1606.05250*.
[67] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for squad," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (Volume 2: Short Papers)*, 2018, pp. 784–789.
[68] L. Cui, Y. Wu, S. Liu, Y. Zhang, and M. Zhou, "Mutual: A dataset for multi-turn dialogue reasoning," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 1406–1416.
[69] H. Huang, Y. Qu, J. Liu, M. Yang, and T. Zhao, "An empirical study of LLM-as-a-judge for LLM evaluation: Fine-tuned judge models are task-specific classifiers," 2024, *arXiv:2403.02839*.
[70] Z. Zhuang, P. Li, P. Ma, W. Meng, and S. Wang, "Testing graph database systems via graph-aware metamorphic relations," *Proc. VLDB Endowment*, vol. 17, no. 4, pp. 836–848, 2023.
[71] Z. Q. Zhou, S. Xiang, and T. Y. Chen, "Metamorphic testing for software quality assessment: A study of search engines," *IEEE Trans. Softw. Eng.*, vol. 42, no. 3, pp. 264–284, Mar. 2016.
[72] X. Xie, J. W. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen, "Testing and validating machine learning classifiers by metamorphic testing," *J. Syst. Softw.*, vol. 84, no. 4, pp. 544–558, 2011.
[73] W. Wang et al., "MTTM: Metamorphic testing for textual content moderation software," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2387–2399.
[74] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 216–226, 2014.
[75] J. Božić, "Ontology-based metamorphic testing for chatbots," *Softw. Qual. J.*, vol. 30, no. 1, pp. 227–251, 2022.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 13:41:50 UTC from IEEE Xplore. Restrictions apply.